Match-Day Nutrition with QAOA

8-minute video presentation script | 8-part structure

0:00-0:45 Part 1 | Introduction

Hello everyone. Our project is called "Match-Day Nutrition with QAOA."

We ask a simple optimization question: how many servings of eight dining-hall foods should a squash player eat on match day?

The goal is to approach several nutritional targets while eating exactly fourteen servings.

We first solve the problem using classical brute force. Then, we convert it into a QUBO and an Ising Hamiltonian and solve it approximately using QAOA. Finally, we investigate how different nutrient priorities change the optimal menu.

0:45-1:35 Part 2 | Data and Variables

Our dataset contains eight foods: eggs, oatmeal, chicken, rice, vegetables, bananas, peanut butter, and pizza.

For each food, we include calories, protein, carbohydrates, fat, potassium, and sodium. The data comes from USDA FoodData Central.

The variable n_i represents the number of servings of food i. Each food can have zero, one, two, or three servings.

$$n_i \in \{0, 1, 2, 3\} \quad \text{and} \quad \sum_i n_i = 14$$

Our example targets include 2,600 calories, 150 grams of protein, and 320 grams of carbohydrates.

Objective and Classical Benchmark

Defining the problem and establishing the exact answer

1:35-2:30 Part 3 | Cost Function

The cost function measures the quality of each menu.

For every nutrient, we calculate the relative difference between the actual amount and its target. We square this difference, so both exceeding and missing the target receive a penalty.

Each nutrient has a weight that represents its importance. Sodium is handled differently: instead of matching a target, the model gives a larger cost to menus with more sodium.

We also add the following penalty to enforce the fourteen-serving requirement:

$$\text{lambda } (\sum_i n_i - 14)^2$$

The best menu is the one with the lowest total cost.

2:30-3:25 Part 4 | Classical Brute Force

We first solve the problem exactly using classical brute force.

Each of the eight foods has four possible serving counts, so there are $4^8 = 65{,}536$ possible menus. A laptop evaluates all of them in approximately 0.02 seconds.

For the performance-focused profile, the optimal menu contains three servings each of oatmeal, chicken, rice, and banana, plus two servings of peanut butter.

It provides 154 grams of protein, 80 grams of fat, and only 779 milligrams of sodium.

This exact result is used as the ground truth for evaluating QAOA.

Quantum Encoding and Algorithm

Moving from the classical objective to a parameterized quantum circuit

3:25-4:25 Part 5 | QUBO and Hamiltonian

To use QAOA, we convert the problem into a Quadratic Unconstrained Binary Optimization problem, or QUBO.

Since each food can have zero to three servings, we encode it using two binary variables:

$$n_i = x_{i,0} + 2x_{i,1}$$

Eight foods therefore require sixteen qubits.

After expanding the cost function, Qiskit converts the linear and quadratic terms into an Ising Hamiltonian containing Z and ZZ operators.

We calculate the ground state and confirm that it matches the brute-force optimum. This shows that our quantum encoding correctly represents the original problem.

4:25-5:25 Part 6 | QAOA Method

QAOA alternates between a cost layer and a mixer layer.

The cost layer uses the nutrition Hamiltonian to favor lower-cost solutions. The mixer layer allows the circuit to explore different menus.

We test three circuit depths: p = 1, p = 2, and p = 3.

A classical COBYLA optimizer adjusts the QAOA parameters to minimize the expected energy. We also use the optimized parameters from a shallow circuit to initialize the next, deeper circuit.

Increasing p can improve the solution, but it also makes the optimization more computationally expensive.

Results and Interpretation

What improved, what did not, and why the objective still matters

5:25-6:40 Part 7 | Results

QAOA performs better as the circuit depth increases.

For a random guess, the probability of producing a state in the best 1 percent is 1 percent. For QAOA, it increases from 2.8 percent at p = 1 to 4.3 percent at p = 3.

The probability of satisfying the fourteen-serving constraint also increases from 10.3 percent for random guessing to 43.4 percent at p = 3.

QAOA does not measure the exact optimum during 4,096 samples. However, its best sampled menu ranks sixth among all 65,536 possible states.

Therefore, QAOA finds a very good solution, although not the exact best one.

6:40-8:00 Part 8 | Conclusion

The definition of "optimal" depends strongly on the chosen weights.

When sodium is ignored, the optimal menu contains three pizza slices. When the sodium weight increases, pizza quickly leaves the optimal menu.

The constraint penalty is also important. If it is too small, the solution violates the fourteen-serving requirement.

For this small problem, classical brute force is much faster, so we do not claim a quantum speedup.

Instead, this project shows how QAOA can model a constrained, human-centered optimization problem - and how human choices determine what the algorithm considers optimal.

Thank you.